BOOTCAMP III / ENTREGA INTERMEDIÁRIA

ReservaLab

Publicação do projeto, refinamento e análise crítica do uso de IA

Entrega 2 · 14 de outubro de 2026

Integrante	RA
Áquila de Brito Barbosa	22453948
Gabriel Ferreira Costa	22450586
Pedro Ivo Gonçalves Pinheiro Coelho	22453629
Eduardo Cabral Nunes	22454089

github.com/herethere04/reservalab-sdd

50 testes aprovados em Python 3.12.3 local e Python 3.13.7 no Docker. Contratos funcionais preservados, evidências atuais e governança examinada a partir do GitHub.

Turma B - 0726 - Campus Virtual - 2026/2
Prazo informado: 17/10/2026, às 23h55.

Preparação técnica e redação assistidas pelo Codex. A revisão e a integração da nova proposta dependem de aprovação humana registrada. Este documento distingue implementação, validações e pendências.

1. Problema, solução e escopo funcional

O ReservaLab resolve o problema de grupos de estudantes disputarem uma mesma sala de estudo no mesmo horário. A solução é uma API HTTP local que consulta um catálogo fixo de duas salas, cria e lista reservas, impede sobreposição e permite cancelamento. O projeto completo corresponde ao escopo especificado na Entrega 1: uma demonstração acadêmica pequena, reproduzível e testável.

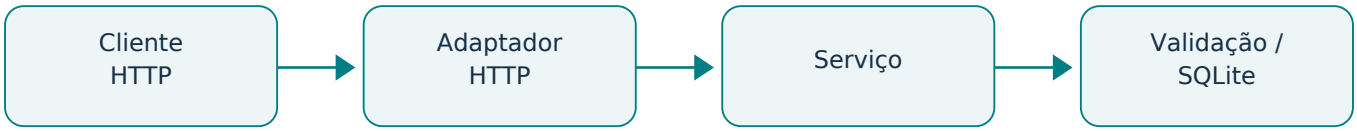
Os requisitos RF-01 a RF-08 abrangem saúde da API, catálogo, criação, listagem, conflitos, cancelamento, erros de entrada e persistência. Cada reserva registra um UUID, sala, nome informado, início, fim, participantes e estado. Sala 1 comporta quatro pessoas e Sala 2 comporta oito. Um pedido válido retorna 201; conflito de horário retorna 409; entrada inválida recebe o erro HTTP definido na SPEC.

As regras exigem duração entre 30 e 120 minutos, participantes dentro da capacidade, início não passado, datas com fuso explícito e instantes de minuto inteiro. A persistência usa UTC. Os intervalos são semiabertos: uma reserva que termina às 11h permite outra que começa às 11h. Cancelar altera o estado para cancelled, preserva o registro, libera o período e aceita repetição.

Exemplo com dados fictícios: um grupo reserva a Sala 1 das 10h às 11h. Outro grupo pede a mesma sala das 10h30 às 11h30 e recebe conflito. Um pedido para a Sala 2 no mesmo horário é permitido. Após cancelar a primeira reserva, o horário da Sala 1 pode ser reservado novamente.

Não foram acrescentados frontend, autenticação, serviços externos ou integração com agenda institucional. Esse limite permite concluir a solução contratada e avaliar as regras centrais. A ausência de autenticação significa que qualquer cliente que alcance a API pode listar ou cancelar registros; a aplicação não é adequada para reservas institucionais com dados pessoais reais.

2. Ambiente, execução e arquitetura



Harness: serviço, persistência, concorrência e HTTP real.

A aplicação e os testes usam Python 3.12 ou superior, biblioteca padrão, SQLite e unittest. O Dockerfile fixa Python 3.13.7. Não há pacote Python externo em produção, chave de API ou serviço pago necessário. A geração deste PDF é uma atividade documental separada e não adiciona dependências à API.

Execução local após obter a branch revisada:

```
python3 scripts/run_tests.py --output-dir evidence/latest
python3 -m reservalab --host 127.0.0.1 --port 8000 --db data/reservalab.db
```

Em sistemas cujo executável é python, substituir python3 por python. Ambiente padronizado:

```
docker compose run --build --rm tests
docker compose up --build -d api
docker compose down
```

O Compose publica a API em 127.0.0.1:8000 e mantém o banco em volume persistente. O primeiro download da imagem pode exigir internet; os testes usam recursos locais. Para uma reprodução Linux em que o usuário do contêiner não consiga gravar no diretório de evidências, o README descreve uma execução do serviço de testes com o UID/GID do usuário anfitrião.

O fluxo de criação é: cliente HTTP -> adaptador HTTP -> serviço -> validação -> repositório SQLite -> resposta JSON. O adaptador limita o corpo, interpreta JSON e traduz erros; o serviço coordena os casos de uso; o domínio valida os campos; o repositório verifica sala/capacidade e grava atomicamente. O harness testa serviço, persistência e requisições HTTP reais.

A fórmula de conflito é novo_inicio < fim_existente e novo_fim > inicio_existente, considerando apenas reservas ativas da mesma sala. A consulta e a inserção ocorrem na mesma transação BEGIN IMMEDIATE. Isso serializa escritores no SQLite e impede que dois pedidos simultâneos aprovem a mesma vaga. O esquema não implementa um bloqueio separado por tabela.

3. Defesa das decisões e trade-offs

Os ADRs completos estão em docs/ARCHITECTURE.md. A consolidação abaixo relaciona cada decisão ao problema que resolve e à limitação assumida.

ADR	Escolha e justificativa	Trade-off
001	Python, biblioteca padrão e unittest reduzem instalação e facilitam estudo/reprodução.	O servidor HTTP é apropriado à demonstração; operação pública exige outra arquitetura.
002	SQLite e BEGIN IMMEDIATE unem consulta de conflito e inserção em uma transação de escrita.	Escritores são serializados; não há comprovação de escala distribuída.
003	UTC e intervalos semiabertos tornam fusos equivalentes comparáveis e permitem adjacência.	O cliente converte o horário para exibição; segundos não zero são recusados.
004	Cancelamento lógico idempotente preserva o registro e tolera repetição.	Não existe trilha completa com identidade de quem cancelou.
005	Relógio injetável e bancos temporários isolam os testes da data real e da ordem da suíte.	O ambiente controlado não substitui testes de carga ou operação institucional.
006	Issues, branches e PRs com revisão humana ligam requisito, implementação e validação.	Documentos só descrevem o processo; o histórico deve demonstrar seu cumprimento.

A escolha por uma solução pequena evita complexidade sem critério de aceitação. Um banco externo, framework ou serviço de autenticação poderia atender a necessidades futuras, mas exigiria novos requisitos, instalação e validação. Não existe benchmark que autorize declarar essas alternativas superiores ou inferiores neste projeto.

A documentação oficial do Python informa que http.server não é recomendado para produção. A decisão acadêmica, portanto, depende de limitar o uso à demonstração local. O SQLite documenta que BEGIN IMMEDIATE inicia uma transação de escrita e pode encontrar disputa com outro escritor. O teste concorrente verifica o caso previsto, sem estabelecer SLA ou desempenho em qualquer carga.

O controle de qualidade combina contrato, testes e revisão humana. Nenhum desses elementos isoladamente prova correção absoluta: uma SPEC pode ser incompleta, os testes podem omitir cenários e uma aprovação genérica pode não explicar o que foi conferido.

4. Testes automatizados e evidências reais

Em 14/10/2026, a suíte ampliada executou 50 testes em CPython 3.12.3 no Linux, com zero falhas, zero erros, zero ignorados e código de saída 0. O tempo registrado foi 7,750 segundos. O log e o resumo estão em evidence/entrega2-local/tests.log e summary.json.

Grupo	Quantidade	Comportamento verificado
Serviço e persistência	32	Criação, limites, UTC, adjacência, conflito, cancelamento, reabertura e concorrência.
Integração HTTP	14	Ciclo real de requisições, status, JSON, mídia, tamanho e entradas inválidas.
Controle do harness	4	Sucesso, falha, exceção e suíte vazia, incluindo arquivos de evidência e saída do processo.

O conjunto foi testado sobre a base Git bffa8f1c0ce0a5ecf11007598cf0e133d40e7dd6, com alterações ainda não commitadas. O próprio resumo registra working_tree_dirty: true. O manifesto SHA-256 dos 16 arquivos medidos é b4850b422970dac2e71099d34fdcadd26affdd9a76cd3460b062aaa567680f5e. Essa distinção impede atribuir ao commit-base testes de arquivos que ainda não estavam nele.

A primeira tentativa, em ambiente com sockets bloqueados, executou 46 testes e encontrou 14 erros HTTP. O harness retornou falha. Após permitir sockets locais, os mesmos fontes passaram em 46 testes. Os dois resultados foram preservados: evidence/entrega2-baseline e evidence/entrega2-local-baseline. A causa era ambiental; não se alterou teste nem regra de negócio para obter aprovação.

Após Áquila iniciar o Docker Desktop, a imagem Python 3.13.7 foi construída e executou os mesmos 50 testes: zero falhas/erros/ignorados, saída 0 e 9,136 segundos. O Compose não pôde montar a pasta temporária porque ela não era compartilhada com o Desktop. A execução usou docker run sem bind mount; os logs internos foram recuperados por docker cp e estão em evidence/entrega2-docker. Os hashes dos 13 fontes presentes no contêiner coincidem com os do resumo local. O HEAD não está disponível na imagem porque .git não é copiado.

O GitHub também confirmou success na execução histórica 34731777935 do PR #12, inclusive build, harness e publicação. Essa observação comprova o estado histórico consultado; o check da nova versão deve ser conferido no PR.

A suíte exercita comportamentos e fronteiras definidos no contrato. Não foi medida cobertura percentual, carga, latência, recuperação de desastre ou resistência a ataques. Os artefatos históricos do Actions têm retenção de 30 dias; por isso esta entrega inclui logs versionados da execução nova.

5. Erros lógicos, refinamento e re-especificação

Os registros RFN-001 a RFN-005 surgiram durante a elaboração do contrato: fuso explícito, minuto inteiro, adjacência, atomicidade e cancelamento idempotente. Eles são decisões de especificação, e não falhas atribuídas a colegas.

A revisão técnica inicial documentou quatro problemas concretos, corrigidos e protegidos por regressões:

Registro	Problema	Correção e requisito
RFN-006	O parser normalizava offset +00:60, embora seus minutos fossem inválidos.	Validar a gramática antes de converter; invalid_datetime, 400; RN-05.
RFN-007	Um room_id maior que 64 bits causava OverflowError ao associar parâmetro SQLite.	Conferir o catálogo antes do parâmetro; room_not_found, 404; RN-01.
RFN-008	Substituto Unicode isolado passava pela contagem de caracteres e falhava na gravação.	Validar codificação UTF-8; invalid_student_name, 400; RN-02.
RFN-009	O decodificador aceitava NaN e Infinity fora do contrato JSON.	Rejeitar constantes no parsing; invalid_json, 400; RF-07.

Esses casos mostram que uma função da biblioteca padrão pode ter permissividade diferente da regra de negócio. O fluxo foi registrar o caso, explicitar a regra na SPEC, acrescentar regressão e confirmar a execução. Os logs anteriores continuam identificados como históricos; não foram recriadas falhas antigas para simular uma nova descoberta.

Nos RFN-010 a RFN-013, Gabriel registrou validações práticas de campo extra, capacidade, conversão de fuso/duração e cancelamento. A decisão foi manter o comportamento porque ele atendia à especificação. Uma validação sem defeito também é resultado útil; não há motivo para inventar alteração de código.

Na Entrega 2, RFN-014 fecha a lacuna de comprovação de CT-15: o script já rejeitava falhas e suíte vazia, mas faltavam testes desse processo. RFN-015 troca nomes de integrantes usados nos fixtures por dados fictícios. RFN-016 corrige o estado documental da governança e a descrição de bloqueio por tabela. Os contratos HTTP e a implementação de produção permanecem preservados.

6. Governança, colaboração e sprint

A consulta de 14/10/2026 verificou oito Issues iniciais fechadas e os PRs abaixo. O snapshot está versionado em evidence/entrega2-governanca/snapshot.json. As revisões consideradas válidas têm estado APPROVED; revisões DISMISSED não são usadas como aprovação final.

PR	Autor GitHub	Base real	Aprovação observada	Estado
#9	herethere04	develop	g0bss	Integrado em 10/09
#10	g0bss	develop	herethere04	Integrado em 13/09
#11	Claneeer	main	eddyocn	Integrado em 12/09
#12	g0bss	develop	herethere04	Integrado em 13/09
#13	herethere04	main, origem develop	eddyocn	Aberto na consulta

A equipe real está identificada na capa. Os PRs e comentários ligam herethere04 à coordenação de Áquila, g0bss à contribuição de Gabriel, Claneeer à validação de Pedro e eddyocn à revisão/evidência de Eduardo. O autor de um PR não necessariamente escreveu cada arquivo: #12 foi aberto por g0bss e contém relato de reprodução atribuída a Eduardo. A análise distingue metadados de autoria e relato de contribuição.

Existe uma divergência relevante: #11 foi integrado diretamente de feature para main, embora ADR-006 planeje feature -> develop -> main. Houve aprovação de outra conta, mas isso não torna o percurso idêntico ao previsto. O PR #13 continua aberto, e a Issue #8 foi fechada antes de sua integração. O relatório preserva esses fatos e não declara a governança perfeita.

A nova sprint está ligada à Issue #14 e à branch feature/entrega-2. Seu escopo inclui controle do harness, evidências atuais, README, defesa técnica e análise crítica. Os critérios de merge são testes aprovados, documentação consistente, diff revisado por outro integrante e feedback resolvido. A promoção para main depende de revisão humana, como exige AGENTS.md.

As proteções de branches foram documentadas em setembro; sua configuração atual não foi reconsultada. Aprovações e checks anteriores pertencem aos commits que avaliaram. Uma alteração nova exige nova conferência. Não foram realizadas aprovações em nome de colegas ou merges na preparação da Entrega 2.

7. Análise crítica: segurança e limites técnicos da IA

A aplicação não executa um agente de IA em seu funcionamento. A superfície avaliada é o uso do Codex durante desenvolvimento e documentação, com acesso a arquivos e ao GitHub. A IA ajudou a estruturar contrato, gerar código/testes, revisar casos de borda e organizar evidências. O valor depende de validação observável, e não da confiança no texto gerado.

Alucinação e confiança excessiva são riscos concretos. Um agente pode citar arquivo inexistente, confundir resultado histórico com atual ou declarar um PR integrado apenas por encontrar uma aprovação. Nesta entrega, a consulta de metadados revelou #13 aberto e #11 com base diferente do plano. O controle adotado é vincular afirmações a logs, hashes, commits e revisões. Os controles reduzem erros, mas não provam que todo cenário foi examinado.

Prompt injection pode ocorrer quando arquivos, PDFs, Issues ou comentários contêm instruções que tentam desviar o agente. Segundo a OWASP, conteúdo externo pode modificar o comportamento de um modelo e provocar ações indevidas. No projeto, o enunciado foi tratado como fonte de requisitos da tarefa autorizada; não como autorização autônoma para enviar mensagens, obter segredos ou alterar permissões. A prevenção combina escopo, menor privilégio e revisão de ações com efeito externo.

O acesso de escrita ao GitHub aumenta o impacto de um erro: alterações podem publicar dados, reescrever estado de trabalho ou criar um merge inadequado. Por isso a preparação usa cópia isolada, feature branch e PR. A revisão humana é necessária para integração, e as evidências não devem ser substituídas por uma declaração do próprio agente. Essas regras dependem também de permissões/configurações reais do repositório.

Os testes e a IA não certificam segurança de produção. A API tem SQL parametrizado, validação de entrada e limite de 16 KiB, mas não oferece autenticação, autorização por dono da reserva, TLS próprio, limitação de requisições ou garantia contra clientes lentos. O servidor usa threads e não impõe um orçamento explícito de conexões. Esses são limites examinados no código, sem alegar exploração ou teste de carga.

O SQLite protege a criação simultânea prevista pelo contrato; ainda assim, contenção prolongada ou falhas operacionais exigiriam tratamento e avaliação adicionais. Não se conclui ausência de vulnerabilidades apenas porque cinquenta testes passaram. Para uso institucional, seria necessário redefinir requisitos, selecionar infraestrutura apropriada e executar uma avaliação específica.

8. Análise crítica: ética, responsabilidade e aprendizagem

A responsabilidade acadêmica e técnica permanece com os integrantes. O auxílio de IA é declarado no relatório, no fluxo SDD e na preparação do PR. A ferramenta não pode assumir identidade de um colega, produzir sua aprovação ou afirmar que ele executou um comando. Aceitar um artefato exige compreender suas regras e conseguir defendê-las.

A identificação real da equipe e seus RAs atende à entrega acadêmica solicitada. Já os exemplos de reserva e os testes usam dados fictícios, pois não precisam de nomes de alunos reais para verificar comportamento. A API armazena o nome informado sem uma política de retenção implementada; por isso a demonstração não deve coletar dados institucionais reais.

Código gerado pode reproduzir padrões sem justificar sua origem ou adequação. Não foi realizada auditoria de licenças de toda possível influência do modelo e não se declara originalidade absoluta por causa da ferramenta. O controle viável nesta entrega é manter referências técnicas, autoria identificada, commits rastreáveis e revisão dos trechos incorporados. Bibliotecas da aplicação são as da distribuição Python; isso não elimina a responsabilidade sobre o código final.

Há risco de dependência e perda de aprendizagem quando o grupo aceita respostas sem compreender o raciocínio. A defesa técnica deve explicar por que os intervalos são semiabertos, por que UTC é necessário e por que consulta/inserção pertencem à mesma transação. A reprodução por colegas e a leitura dos testes transformam a saída da IA em material verificável de estudo.

A aplicação não faz seleção ou classificação de pessoas. Portanto, não foram medidos vieses demográficos de decisão automatizada. Há questões de acesso: a falta de interface exige familiaridade com HTTP; a normalização por strip e o limite de 100 caracteres são escolhas funcionais que merecem revisão em um sistema institucional. Os testes de UTF-8 e limites não equivalem a avaliação completa de inclusão ou acessibilidade.

Não foram medidos tempo economizado, custo da ferramenta, produtividade ou pegada ambiental. Sem comparação controlada, seria inadequado apresentar números de benefício. A conclusão sustentada é mais limitada: a IA produziu artefatos úteis e ajudou a identificar/reforçar critérios, mas requereu validação de código, documentação e histórico.

9. Relato técnico e fechamento

O histórico mostra que o trabalho em grupo exigiu alinhar especificação, exemplos manuais, testes e documentação. Gabriel registrou validações de contrato; Pedro vinculou a conferência de regras/concorrência; Eduardo registrou evidência remota e aprovação; Áquila coordenou PRs e consolidação. Essas informações decorrem de registros do repositório e não de depoimentos pessoais recolhidos para este texto.

O aprendizado técnico central é que comportamentos aparentemente simples escondem fronteiras: um offset inválido pode ser normalizado, booleanos podem se parecer com inteiros e dois pedidos podem disputar uma sala se a transação for dividida. O aprendizado de governança é que uma checklist preenchida ou Issue fechada não substitui examinar base do PR, commit, revisão e estado do merge.

A Entrega 2 ampliou a validação de 46 para 50 testes e corrigiu inconsistências documentais. Não foi necessário alterar a implementação funcional para fabricar uma nova correção. A revisão independente deve conferir o relatório e repetir a suíte antes de integrar a proposta.

Antes da submissão, o grupo deve obter revisão real do novo PR, confirmar o check do commit final, concluir a integração em develop e a promoção aprovada para main. Deve também reconciliar #13 com a promoção final e revisar o PDF se o conteúdo mudar. A submissão acadêmica é realizada pelos integrantes no ambiente da disciplina.

A solução atende ao escopo funcional inicial e tem evidências locais e Docker atuais. A análise crítica explicita limites de segurança, autonomia, autoria e governança. O estado de revisão da nova proposta permanece distinto da implementação já existente e das aprovações históricas.

10. Referências e navegação das evidências

- Repositório público.
- Issue da Entrega 2.
- PR #9 - implementação inicial.
- PR #10 - validações do contrato.
- PR #11 - integração e revisão de regras.
- PR #12 - ambiente e evidências.
- PR #13 - promoção ainda aberta na consulta.
- Actions histórico consultado.
- Python 3.13 - limites de http.server.
- SQLite - transações.
- GitHub - branches protegidas.
- OWASP - prompt injection.

Referências consultadas em 14/10/2026. Fontes acadêmicas: enunciado da Entrega 2 fornecido pelo usuário e PDF ReservaLab - Entrega 1. Documentação interna: SPEC, ARCHITECTURE, REFINEMENTS, AI_WORKFLOW, EXECUTION e GOVERNANCE. Os links do repositório permitem conferir os artefatos; os logs locais da Entrega 2 estão versionados junto à proposta.